# Loan Default Prediction

## Comparing Classic ML and Neural Networks under Class Imbalance

Minwoo Yoo  Nathaniel Badalov

Final Project Group 15 · Home Credit Default Risk (Kaggle)

# Problem & Motivation

**Task** Binary classification — predict loan default ($\texttt{TARGET} \in \{0, 1\}$)

**Data** 307,511 applicants $\times$ 122 features (Kaggle)

**Challenge** only **8.07%** default $\Rightarrow$ severe class imbalance

**Approach** 5 models (3 classic ML + 2 NN) + imbalance handling on best NN

**Metric** ROC AUC (accuracy is misleading on imbalanced data)

*Research question: do classic ML and NN respond differently to imbalance handling?*

# Dataset · Home Credit Default Risk

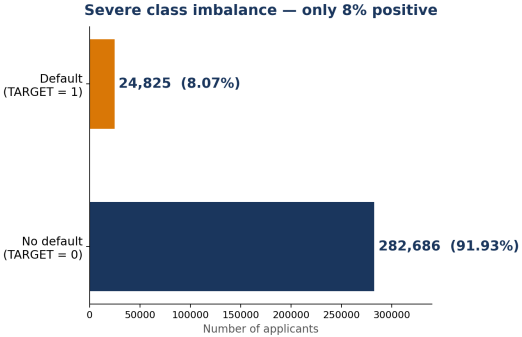
**Structure** Tabular — row = one applicant

**Obs.** 307,511 train / 48,744 test

**Features** 122 (102 numeric, 16 categorical)

**Target** TARGET $\in \{0, 1\}$; 1 = default

**Imbalance** **8.07%** default rate




## Strongest signals

- EXT_SOURCE_1/2/3 external credit scores
- Demographics, employment, credit bureau

# Classic ML Models

| Model | Best hyperparameters | Val AUC |
|---|---|---|
| Logistic Regression | `C`=0.1; `class_weight=balanced` | 0.7487 |
| Random Forest | `n_estimators`=200; `leaf`-regularized; `balanced` | 0.7470 |
| HistGradientBoosting | `lr`=0.05; `max_iter`=200; `leaf`=100 | **0.7595** |

## Key takeaways

- HGBC wins by ∼1 AUC point — boosted trees dominate tabular.
- `class_weight='balanced'` on LR / RF handles imbalance in the loss.
- RF overfits at `leaf=1`: train 0.96 vs val 0.747.

# Neural Network Models

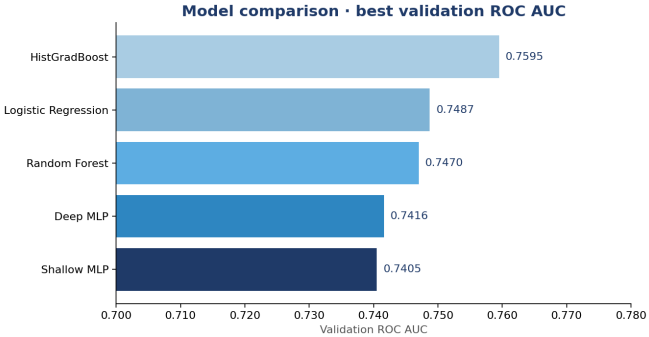
| Architecture | Hidden | Parameters | Val AUC |
|---|---|---|---|
| Shallow MLP | $(64,)$ | $\sim$15.7K | 0.7405 |
| Deep MLP | $(128, 64, 32)$ | $\sim$42K | **0.7416** |

ReLU · Adam · early stopping.
Tuned: `alpha` $\in \{10^{-4}, 10^{-3}\}$, `lr` $\in \{0.001, 0.005\}$.
Best for both: $\alpha = 10^{-3}$, `lr` $= 0.005$.

## Key takeaways

- Depth doesn't help: 0.7405 vs 0.7416 (within noise).
- Tripling parameters bought nothing.
- Both NNs land below the classic learners.

# Combined Leaderboard




| Model | Val AUC |
|---|---|
| HistGradBoost | **0.7595** |
| Logistic Reg. | 0.7487 |
| Random Forest | 0.7470 |
| Deep MLP | 0.7416 |
| Shallow MLP | 0.7405 |

Tree boosting wins by $\sim$1 AUC point.

# Imbalance Handling on the Best MLP

| Method | Thresh. | AUC | PR-AUC | Prec. | Recall | F1 |
|---|---|---|---|---|---|---|
| Baseline | 0.500 | 0.745 | 0.222 | 0.00 | 0.00 | 0.00 |
| SMOTE | 0.500 | **0.648** | 0.134 | 0.16 | 0.21 | 0.18 |
| Threshold tuning $\star$ | 0.164 | 0.745 | 0.222 | 0.24 | **0.40** | **0.30** |

## Key takeaways

- Baseline at 0.5: AUC 0.745 but recall **0%**.
- SMOTE hurts: AUC drops 0.10.
- Threshold tuning at 0.164 lifts recall to **40%**.

*Threshold > resampling.*

# Conclusions

## What we learned

- HGBC wins (val AUC = **0.7595**).
- NN ties LR / RF — depth alone doesn't help on tabular.
- Threshold tuning lifts MLP recall **0%** $\rightarrow$ **40%**; SMOTE hurt AUC.

## Limitations

- Only main `application_*.csv` tables (auxiliary tables would add several AUC points).
- Single train/val split (PredefinedSplit), not full $k$-fold CV.

# Recording

<<INSERT RECORDING LINK BEFORE SUBMISSION>>

*Minwoo Yoo · Nathaniel Badalov*